

Part 1: Teams as the Means of Delivery

Chapter 1: The Problem with Org Charts

"The organizational structure must coordinate accountabilities to support the goals of delivering high-quality, impactful software"

Systems thinking focuses on optimizing for the whole, looking at the overall flow of work, identifying what the largest bottleneck is today, and eliminating it. Then repeat.

Three Organizational Structures in Every Organization:

- Formal Structure (org chart) - facilitates compliance
- Informal Structure - the "real of influence" between individuals
- Value Creation Structure - how work actually gets done based on inter-personal and inter-team reputations

Five Rules of Thumb for Designing Organizations:

- Design when there is a compelling reason
- Develop options for deciding on a design
- Choose the right time to design
- Look for clues that things are out of alignment
- Stay alert to the future

Four Fundamental Team Types:

- stream-aligned
- platform

- enabling
- complicated-subsystem

Three Core Team Interaction Modes:

- Collaboration
- X-as-a-Service
- Facilitating

Conways Law: "Organizations which design systems... are constrained to produce designs which are copies of the communication structures of these organizations"

ie: There is a strong correlation between an Organizations real communications paths (value creation structure) and the resulting software architecture.

"If you have 4 groups working on a compiler, you'll get a 4-pass compiler"

Three Elements of Intrinsic Motivation:

- Autonomy
- Mastery
- Purpose

We need to put the team first, advocating for restricting their cognitive load.

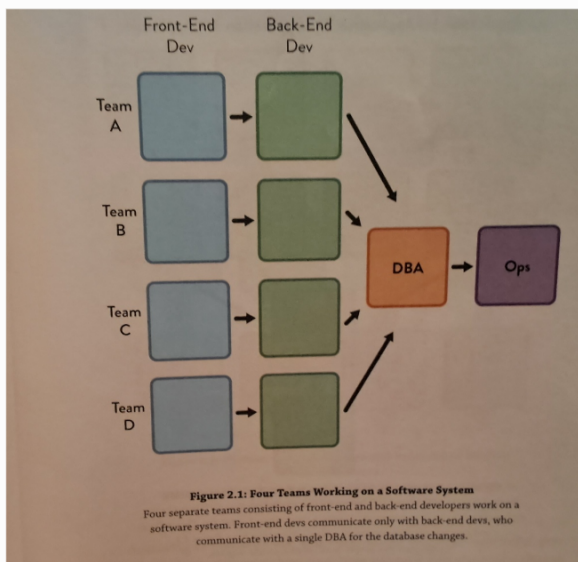
Chapter 2: Conway's Law and Why it Matters

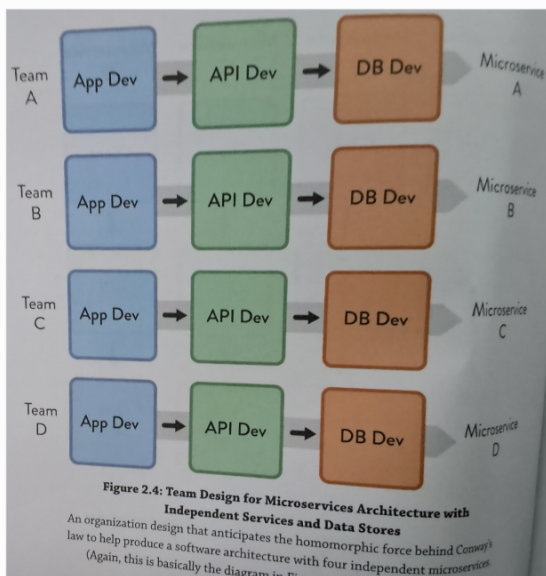
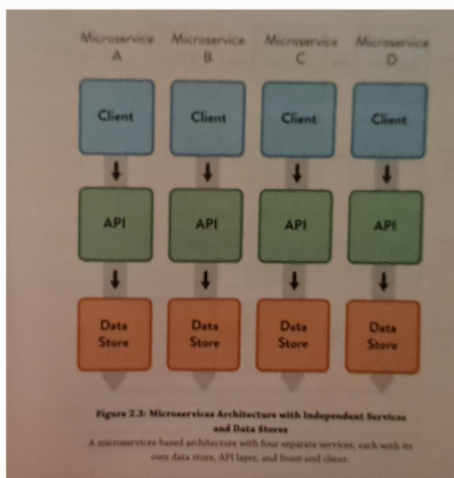
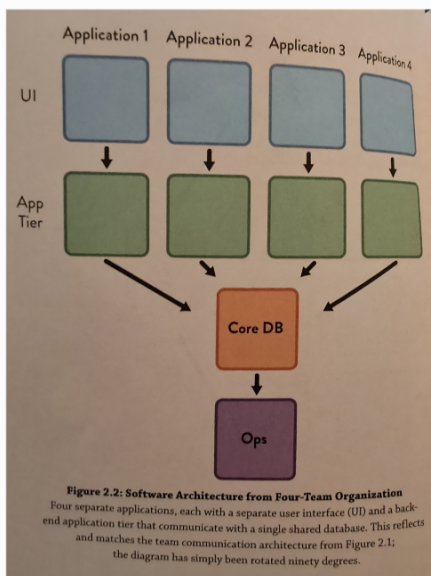
"If the architecture of the system and the architecture of the organization are at odds, and the architecture of the organization wins"

An organization that is arranged in functional silos is unlikely to ever produce software systems that are well architected for end to end flow.

Reverse Conway Maneuver: *Organizations should evolve their team and organizational structure to achieve the desired architecture. The goal is for your architecture to support the ability of teams to get their work done - from design through to deployment - without requiring high bandwidth communication between teams.*

Ex:





"Team assignments are the first draft of the architecture"

Software architecture good practices:

- Loose coupling - components do not hold strong dependencies on other components
- High cohesion - components have clear bounded responsibilities, and their internal elements are strongly related
- Clear and appropriate version compatibility
- Clear and appropriate cross-team testing

"We can also exploit architecture as an enabler of rapid change. We do this by partitioning our architecture to gracefully absorb change."

"If we have managers deciding... which services will be built, by which teams, we implicitly have managers deciding on the system architecture"

More communication between teams is not always better. Focused communication is what is needed. Look for unexpected communications and address the cause. (Kevan: I get the theory behind this, but it makes me feel... icky)

"Managers should focus their efforts on understanding the causes of unaddressed design interfaces... and unpredicted team interactions... across modular systems"

(Kevan: I think this is really getting at how to avoid spaghetti code. If everyone need to communicate with everyone else, when your architecture will become a complicated tangled mess.)

Chapter 3: - Team First Thinking

"Disbanding high-performing teams is worse than vandalism: it is corporate psychopathy."

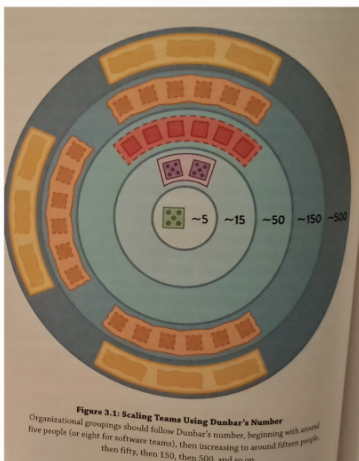
Teams working as a cohesive unit perform far better than collections of individuals for knowledge-rich, problem-solving tasks that require high amounts of information

Google found for their own teams, who is on a team matters less than the team dynamics; and that when it comes to measuring performance, teams matter more than individuals.

Team: a stable group of 5 to 9 people who work toward a shared goal as a unit.

High trust is explicitly valued and designed for. High trust is when enables a team to innovate and experiment.

- Team 5-9 (No more than 15)
- Families ("Tribes") no more than 50 people
- Divisions/Streams/Profit & Loss lines groupings of no more than 150 or 500 people



In high-trust organizations people can switch teams every 9-12 months. In organizations with lower levels of trust people should remain on the same team for 18 months-2 years

Tuckman Teal Performance Model describes how teams perform:

1. Forming: Assembling for the first time
2. Storming: Working through initial differences in personality and ways of working
3. Norming: Evolving standard ways of working together
4. Performing: Reaching a state of high effectiveness

Every part of the software system needs to be owned by exactly one team. This mean there should be no shared ownership of components, libraries, or code. Teams may use shared services at run time, but every running service, application, or subsystem is owned by only one team. Outside teams may submit pull requests, or suggestions for change to the owning team, but they cannot make the change themselves.

(Kevan: I'm pretty sure our team really really doesn't do this at the moment.)

Ownership of code should not be a territorial thing. The team takes responsibiliy for the code, and cares for it, but individual teams members should not feel like the code is theirs to the exclusion of others. The team should view

themselves as stewards or caretakers as opposed to private owners. Think gardeners not police.